

DEVOPS ENGINEER ASSESSMENT

CI/CD Automation for .NET Deployment to Minikube

Azure DevOps · Docker · Kubernetes · Prometheus & Grafana

github.com/Ge0rak/minikube-cicd

What this presentation covers

1

Architecture

How the pieces fit together end to end

2

Core Tasks

Azure DevOps, Docker, Kubernetes, Monitoring

3

Bonus Work

Probes, autoscaling, layered testing, failover design

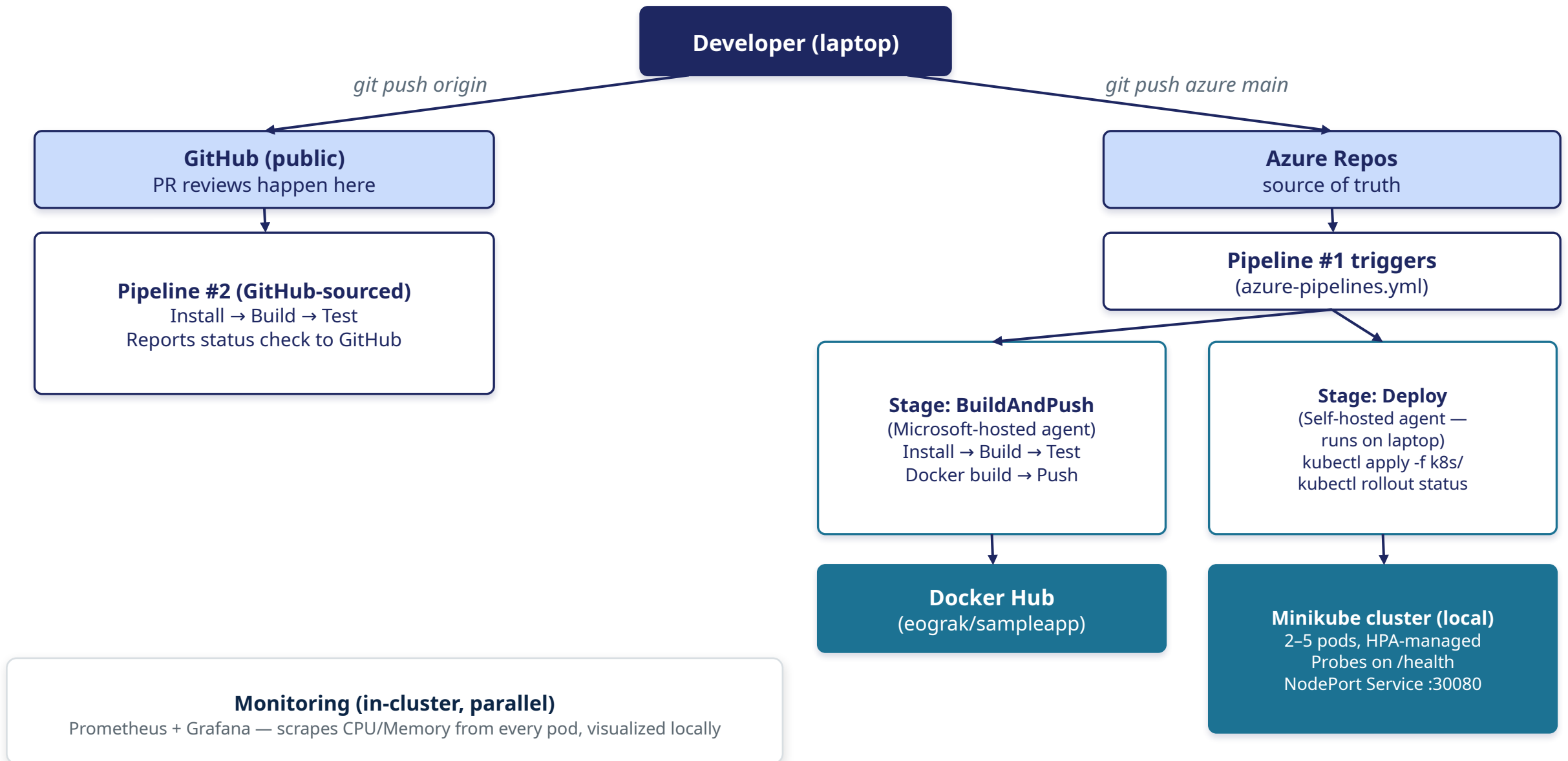
4

Challenges

What went wrong, and what it taught

CI/CD Pipeline Architecture

.NET application — Azure DevOps CI/CD to a Minikube cluster



TASK 1

Azure DevOps Configuration

- Azure DevOps project + Git repo, mirrored to a public GitHub repo for visible commit history
- Minimal ASP.NET Core 10 Web API with /weatherforecast and a custom /health endpoint
- YAML build pipeline: triggers on every commit to main, installs SDK, restores, builds

WHY IT MATTERS

YAML pipelines version the CI/CD definition alongside the code. A feature-branch + PR workflow was kept consistent for the entire project, giving a clean, incremental history.

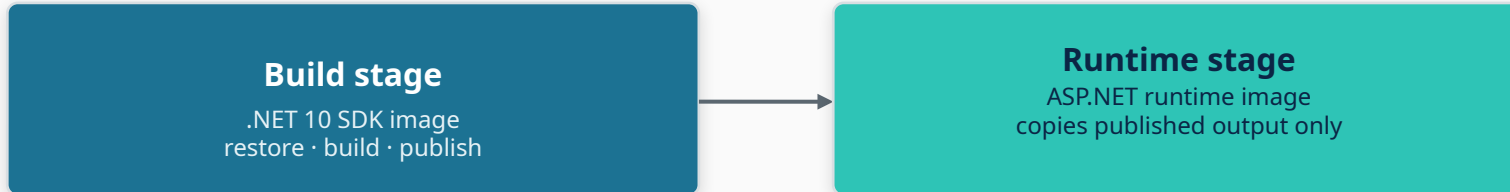
The screenshot displays the Azure DevOps web interface for a project named 'georgioskaramanis'. The left sidebar shows the navigation menu with 'Pipelines' selected. The main area shows a pipeline run for 'minikube-cicd' with the ID '20260902.1'. The pipeline is in a 'Jobs in run' state, and the 'Build and Push' job is currently running. The 'Build' step is highlighted, showing a list of tasks: 'Initialize job', 'Checkout minikube-cic...', 'Install .NET SDK', 'Restore dependencies', 'Build', 'Run tests', 'Build and push Dock...', 'Post-job: Checkout m...', 'Deploy to Minikube', 'DeployToK8s', and 'Finalize build'. The 'Install .NET SDK' task is currently executing, and its log is visible on the right. The log shows the task's description, version, and the steps taken to install the .NET Core SDK version 10.0.400 on a Linux x64 platform.

```
1 ► Condition evaluation
9 Starting: Install .NET SDK
10 =====
11 Task : Use .NET Core
12 Description : Acquires a specific version of the .NET Core SDK from the internet or the local cache and adds it
13 Version : 2.274.2
14 Author : Microsoft Corporation
15 Help : https://aka.ms/AA4xy0
16 =====
17 [Host] Selected Node version: Node24 (Strategy: Node24Strategy)
18 Tool to install: .NET Core sdk version 10.0.x.
19 Found version 10.0.400 in channel 10.0 for user specified version spec: 10.0.x
20 Version 10.0.400 was not found in cache.
21 Getting URL to download .NET Core sdk version: 10.0.400.
22 Detecting OS platform to find correct download package for the OS.
23 /home/vsts/work/_tasks/UseDotNet_b0ce7256-7898-45d3-9cb5-176b752bfea6/2.274.2/externals/get-os-distro.sh
24 get-os-distro: Error: Distribution specific OS name and version could not be detected: UName = Linux
25 Primary:linux-x64
26 Legacy:-x64
27 Detected platform (Primary): linux-x64
28 Detected platform (Legacy): -x64
29 nloading: https://builds.dotnet.microsoft.com/dotnet/Sdk/10.0.400/dotnet-sdk-10.0.400-linux-x64.tar.gz
30 Extracting downloaded package /home/vsts/work/_temp/dee0f55c-8f76-4f52-aba0-967e7122c0b9.
31 Extracting archive
32 /usr/bin/tar xC /home/vsts/work/_temp/98012797-d29e-4f13-836b-481bb198beef -f /home/vsts/work/_temp/dee0f55c-8f76
33
34 Successfully installed .NET Core sdk version 10.0.400.
35 Creating global tool path and pre-pending to PATH.
36 Finishing: Install .NET SDK
```

TASK 2

Docker Image Creation

Docker Image Multi stage build



INFO

- Multi-stage build → small final image, no SDK/compiler.
- Docker@2 task builds, tags with a unique build ID + latest, and pushes to Docker Hub via a secure service connection — no credentials in the YAML.

The screenshot shows the Docker Hub interface for a user named 'eograk'. The left sidebar contains navigation links: 'eograk Your personal account', 'Repositories', 'Hardened Images', 'Collaborations', 'Settings' (with sub-links for 'Default privacy' and 'Notifications'), 'Billing', 'Usage' (with sub-links for 'Pulls' and 'Storage'), and an 'Upgrade subscription' button at the bottom. The main content area is titled 'Repositories' and shows 'All repositories within the eograk namespace.' It includes a search bar, a dropdown for 'All content', and a 'Create repository' button. A table lists the repository 'eograk/sampleapp', which was 'Last Pushed' 'about 2 hours ago', 'Contains' an 'IMAGE', and has 'Public' visibility. The table also shows a 'Scout' status of 'Inactive'. At the bottom right of the table, it indicates '1-1 of 1' items.

Name	Last Pushed ↑	Contains	Visibility	Scout
eograk/sampleapp	about 2 hours ago	IMAGE	Public	Inactive

TASK 3

Kubernetes Deployment

Azure Pipelines

Microsoft-hosted
cloud agent



No network route

Minikube

Local cluster,
laptop-only

SOLUTION: SELF-HOSTED AGENT

A small agent process runs directly on the laptop and polls Azure DevOps for work. The pipeline's Deploy stage routes to this agent specifically, giving it the same local network access kubectl already has.

TWO-STAGE PIPELINE

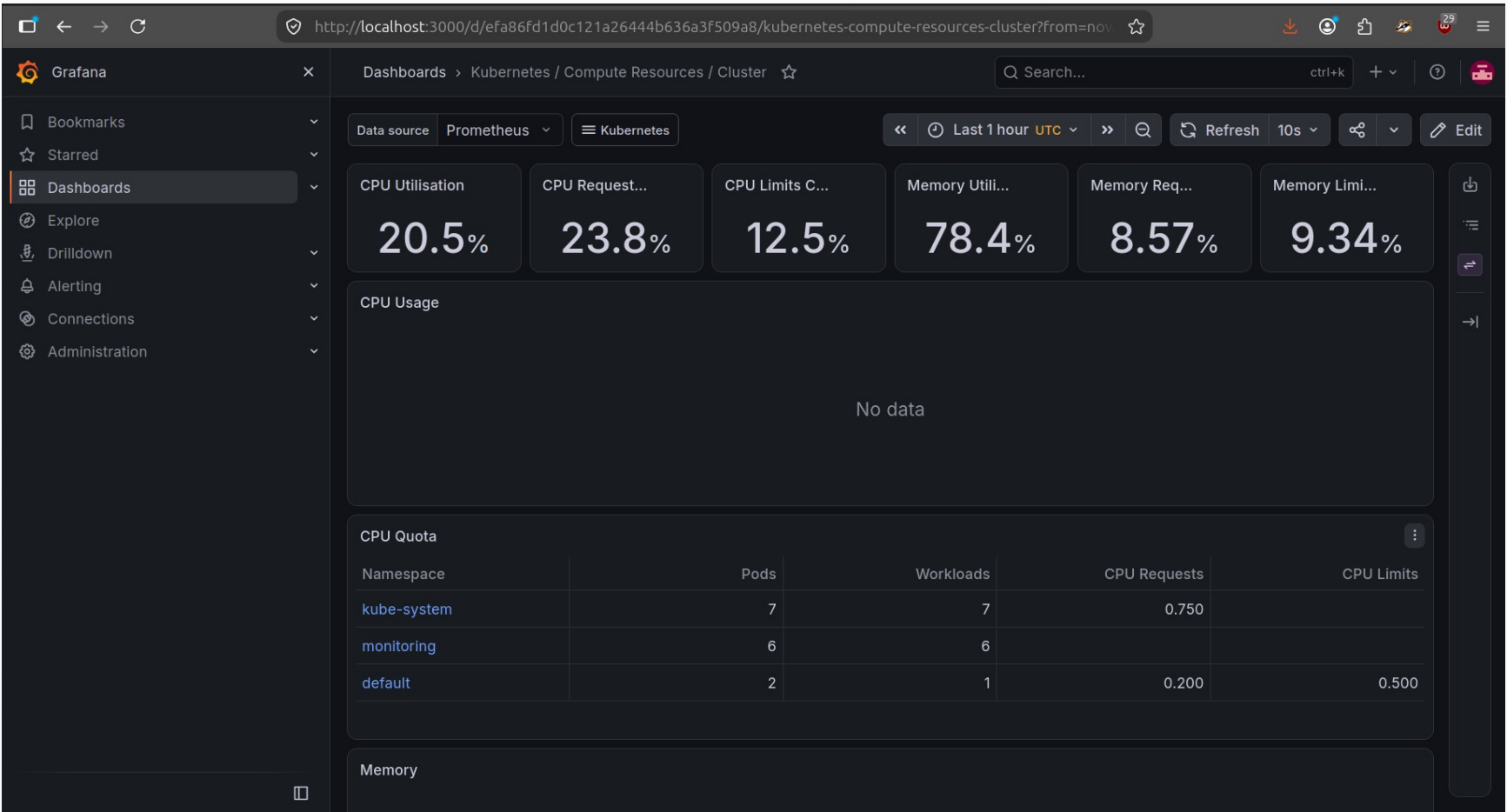
BuildAndPush

Microsoft-hosted agent

Deploy

Self-hosted agent — kubectl apply + rollout status

Monitoring – Grafana Dashboard



117%
Peak CPU
observed under load

2 → 4
Pods scaled up
automatically

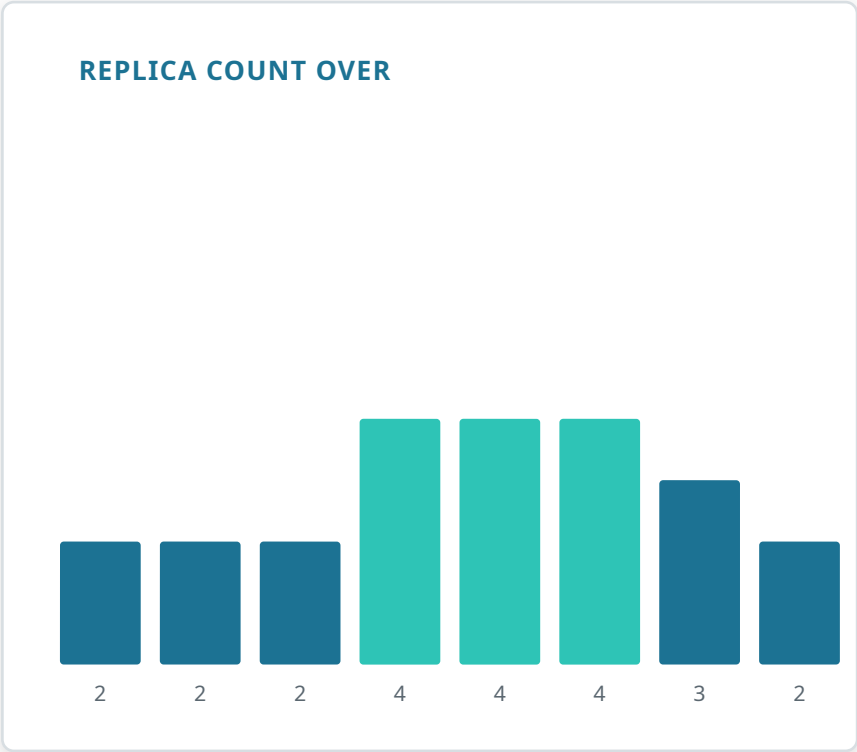
- KPI COVERAGE**
- CPU Saturation **Covered**
 - Memory Usage **Covered**
 - Error Rate (restart count) **Covered**
 - Availability **Covered**
 - Latency / APM tracing **Known gap**

Verified under real, generated load

```
eog@eog-ThinkPad-X270-W10DG:~/GitHub/minikube-cicd$ kubectl get hpa --watch
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 1%/70%, memory: 34%/80%	2	5	2	18m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 22%/70%, memory: 38%/80%	2	5	2	18m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 117%/70%, memory: 38%/80%	2	5	2	19m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 117%/70%, memory: 38%/80%	2	5	4	19m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 62%/70%, memory: 33%/80%	2	5	4	20m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 58%/70%, memory: 33%/80%	2	5	4	21m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 60%/70%, memory: 33%/80%	2	5	4	22m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 53%/70%, memory: 33%/80%	2	5	4	23m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 50%/70%, memory: 33%/80%	2	5	4	24m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 44%/70%, memory: 33%/80%	2	5	4	25m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 1%/70%, memory: 33%/80%	2	5	4	26m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 1%/70%, memory: 33%/80%	2	5	4	27m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 1%/70%, memory: 33%/80%	2	5	4	28m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 1%/70%, memory: 33%/80%	2	5	4	29m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 1%/70%, memory: 33%/80%	2	5	3	29m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 1%/70%, memory: 33%/80%	2	5	3	30m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 1%/70%, memory: 33%/80%	2	5	3	31m
sampleapp-hpa	Deployment/sampleapp-deployment	cpu: 1%/70%, memory: 36%/80%	2	5	2	31m

Terminal-kubectl: Replicas reacting to load generator



Scaled 2→4 within seconds of crossing target|
Gradual step-down 4→3→2 after load stopped.

Two more safety mechanisms, both verified live

HEALTH PROBES



Startup

1s interval, 30s grace — handles slow boots



Liveness

10s interval, 3 fails → restart stuck pods



Readiness

5s interval, 3 fails → pulled from traffic

TESTS RUN IN THREE PLACES

Local

dotnet test, on demand during development

Pre-merge

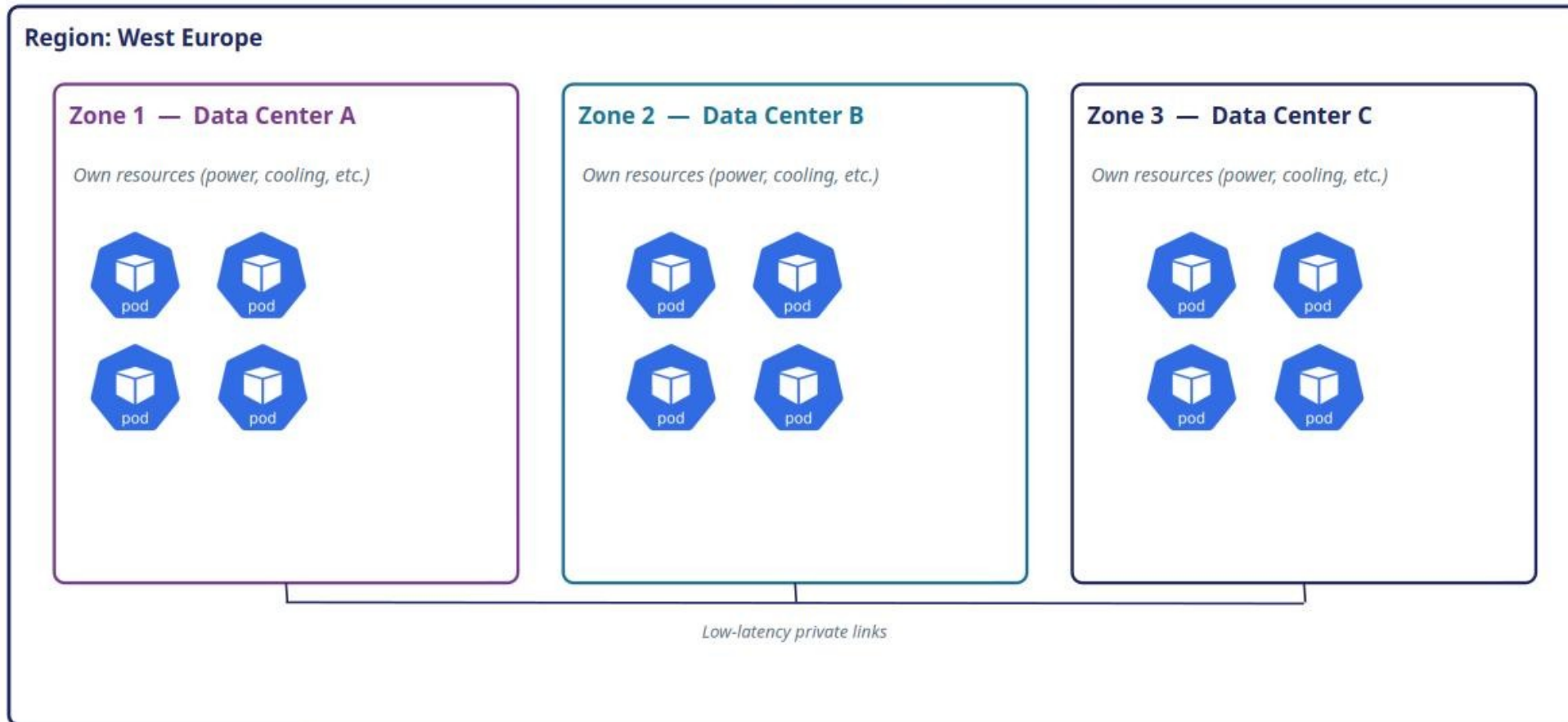
GitHub PR check — second pipeline, GitHub-sourced, blocks a bad merge

Post-merge

Main CD pipeline re-runs tests, blocks Docker build/deploy on failure

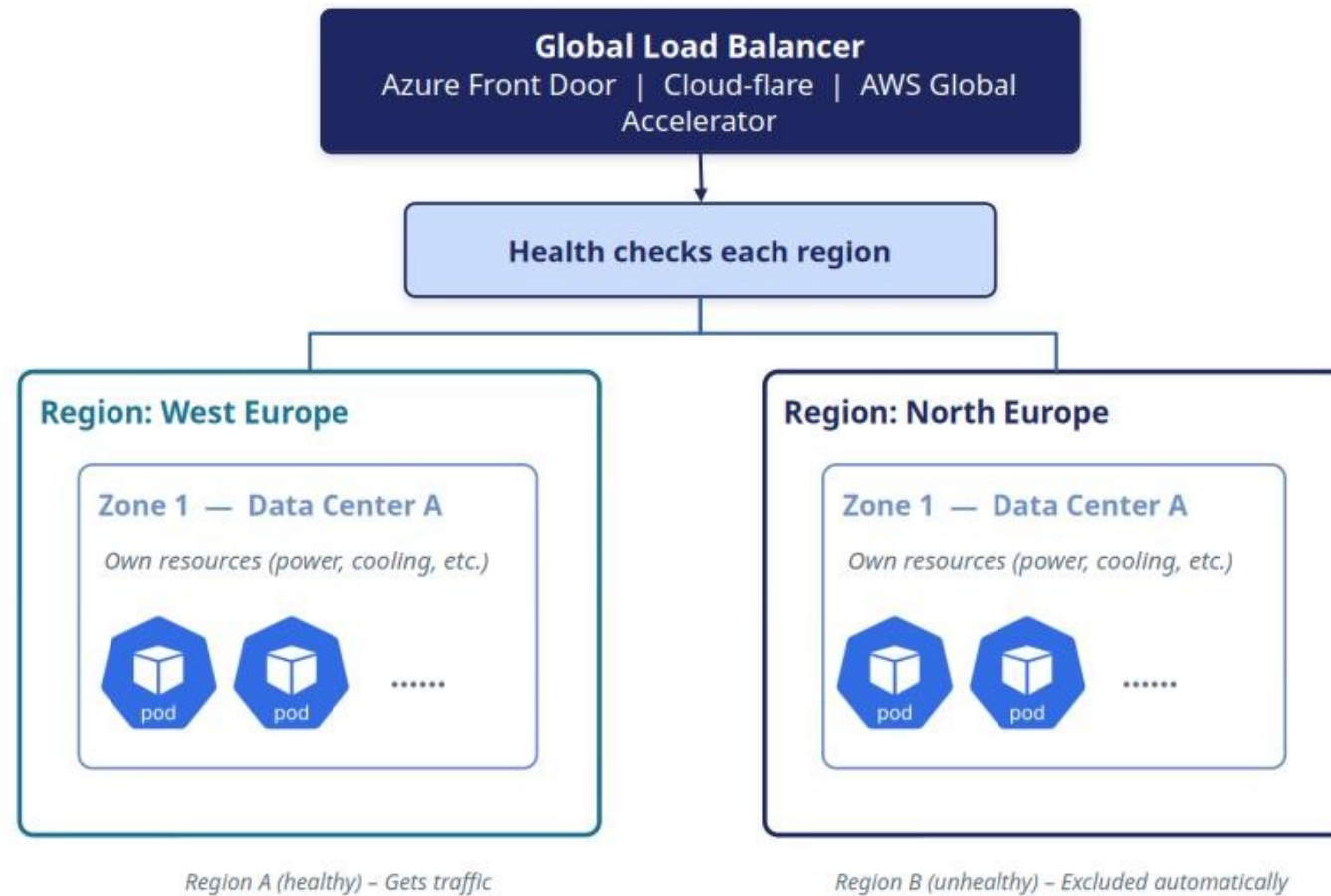
Multi-Zone Redundancy within a Region

Availability-zone isolation keeps pods running if one data center fails



Multi-Region Fail-over Strategy

Global traffic routing across independent Azure regions



Real obstacles, resolved along the way

Docker's apt repo had no Ubuntu 26.04 build yet

Pointed at the 24.04 (noble) repo — fully compatible

.dockerignore omission broke the first Docker build

Local bin/obj artifacts were silently overwriting a fresh restore

Cloud agents can't reach a local-only cluster

Solved with a self-hosted agent running on the laptop

Program class wasn't visible to the test project

Standard fix: `public partial class Program {}` for `WebApplicationFactory`

Thank you

github.com/Ge0rak/minikube-cicd